Future Generation Computer Systems 149 (2023) 637–649

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

SDN/NFV-based framework for autonomous defense against slow-rate DDoS attacks by using reinforcement learning

Noe M. Yungaicela-Naula *, Cesar Vargas-Rosales, Jesús A. Pérez-Díaz

Tecnologico de Monterrey, School of Engineering and Sciences, Monterrey, 64849, Nuevo Leon, Mexico

ARTICLE INFO

Article history:
Received 9 November 2022
Received in revised form 30 May 2023
Accepted 4 August 2023
Available online 7 August 2023

Keywords:
Network function virtualization (NFV)
Moving target defense (MTD)
Reinforcement learning (RL)
Slow-rate DDoS
Software defined networking (SDN)
Zero touch networks and service management (ZSM)

ABSTRACT

The unforeseen and skyrocketed shift in the number of connections to the Internet during the last years has created vast and critical vulnerabilities in networks that cybercriminals have quickly seized to launch high-volume DDoS attacks. Existing tools, such as advanced firewalls or intrusion prevention systems (IPS), cannot handle such an elevated volume of attacks because these solutions are dependent on humans. Therefore, adaptation of the current network security solutions to automated ones is more significant than ever to foster the development of the zero-touch networks and service management (ZSM) paradigm. Building on our preliminary work in this field, in this study, we provide a software-defined networking (SDN)-based framework that automates the detection and mitigation of slow-rate DDoS attacks. The framework uses deep learning (DL) to detect attacks and reinforcement learning (RL) to mitigate them. Furthermore, a network function virtualization (NFV)-assisted moving target defense (MTD) mechanism is included to amplify the effectiveness and flexibility of the solution. The framework is tested on a simulated network using open-source tools, namely Open Network Operating System (ONOS), Containernet, Apache Web Server, and Docker. The source code of a prototype of the framework is shared, which can be used and improved by interested researchers. Finally, the experimental results demonstrate that RL agents learn optimal DDoS mitigation policies in different scenarios and that they quickly adapt to new conditions that vary in short periods of time.

1. Introduction

In recent years, the unexpected and paramount shift in Internet usage has exposed new network vulnerabilities that cybercriminals have exploited to launch more complex and higher-throughput DDoS attacks [1]. In this regard, Kaspersky[1] reported an impressive growth in the number of DDoS attacks in 2021. In particular, the number of attacks in Q4 2021 increased more than 4.5 times against the number of attacks in the same period in 2020.

Several solutions have been proposed for DDoS attacks. On the one hand, many researchers have studied the efficacy of machine learning (ML) and deep learning (DL) models in the detection of complex DDoS attacks [2,3]. In addition, several mitigation solutions have been studied that go from traffic drop strategies to those that use reinforcement learning (RL). The use of RL helps optimize the utilization of network resources during attack mitigation [4]. However, ML- and DL-based solutions need to continuously monitor the network while the attackers need to find a single point of failure.

On the other hand, proactive mechanisms have been developed, which implement various network configurations, making the system more difficult to attack. One example of these mechanisms is moving target defense (MTD), which dynamically configures the network to puzzle the attack surfaces [5]. MTD can be assisted by network function virtualization (NFV) to configure the network's logical structure and the functionality of the different network devices. Therefore, previous work has shown that this combination provides a flexible, scalable, and effective solution against DDoS attacks [6].

Numerous next-generation networks (NGNs), such as IoT, 5G, and 6G, are proliferating, which demand end-to-end automation of proactive attack discovery, application of intelligent techniques, and the achievement of self-sustaining networks [7]. In this respect, according to Hummel et al. [1], when it comes to countermeasure DDoS attacks, solutions based on network flow analysis, as well as next-generation firewalls are preferred among service providers. Nonetheless, these solutions are low-effective as they focus on DDoS attack detection, while mitigation has barely been explored. At this time, automated security solutions that simplify and speed up response times of DDoS mitigation are more important than ever. Such solutions will promote the

* Corresponding author.
E-mail addresses: a00821711@tec.mx, marceloy@ieee.org (N.M. Yungaicela-Naula).

[1] https://securelist.com/ddos-attacks-in-q4-2021/105784/.

https://doi.org/10.1016/j.future.2023.08.007
0167-739X/

advancement of the emerging concept of zero-touch network and service management (ZSM), whose ultimate objective is to enable self-monitored, configured, optimized, and healed NGNs [8,9].

Software-defined networking (SDN) has the potential to provide intelligent incident detection and automated mitigation response [10]. Unlike traditional networking, in the SDN paradigm, the control plane is detached from the network devices, and placed in a centralized agent, named controller [11]. Therefore, SDN grants global monitoring of the network and dynamic configuration of the routes.

In a first attempt to provide an autonomous defense against DDoS attacks, in our previous study in [12], we presented a scalable SDN-based security framework to mitigate slow-rate DDoS attacks. In this framework, a DL-based intrusion detection system (IDS) detects attacks and a deep reinforcement learning (DRL)-based intrusion prevention system (IPS) adaptively learns and executes an optimized mitigation task. Although that previous work constituted an important step towards the security automation, still a further improvement can be made to increase the framework's performance, scalability, and flexibility. For example, in this previous study, the set of actions of the DRL method was limited to blocking and releasing connections, which can be extended by adding more effective DDoS mitigation actions. Additionally, the design of the RL problem and its solution can be simplified to reduce the processing overhead of the controller.

Therefore, in this work, an SDN/NFV-based security framework is presented, which integrates DL, RL, and MTD to obtain an effective and autonomous slow-rate DDoS attack mitigation. NFV technology is used to deploy the MTD mechanism, which allows us the placement of network functions (NFs) on demand, providing flexibility and scalability for the security solution. Further, the design of the RL problem uses a finite number of states, meaning its solution requires low computational resources, which increases the scalability of the proposed framework.

The framework is tested in simulations using open-source and professional tools such as Open Network Operating System (ONOS,[2]) controller, Containernet[3] Apache Web Server, and Docker. The experimental results demonstrate that the framework autonomously deploys an optimized DDoS mitigation policy.

In summary, the contributions of this work are as follows.

1. We provide an automated SDN-based framework that combines intelligent methods, such as DL and RL, and emerging networking paradigms, such as MTD and NFV, to defend the network against slow-rate DDoS;
2. We perform a real-time performance evaluation of the proposed solution in a simulated network using Containernet and the ONOS controller. This assessment demonstrates that the designed RL agents can learn the optimal mitigation policies to be applied in different scenarios and that they can quickly adapt to rapid varying conditions; and
3. We provide the source code for a prototype of the proposed framework.

The remainder of this paper is organized as follows. Section 2 explores similar work and highlights the contribution of our solution. The proposed framework is briefly described in Section 3. The traffic monitoring strategy and the IDS are described in Section 4. Section 5 reports on the design of the MTD mechanism. The RL problem and the solution to mitigate DDoS are presented in Section 6. The experimental results are presented in Section 7. Section 8 discusses the findings of this study. Finally, the conclusion and future research are presented in Section 9.

[2] https://www.opennetworking.org/onos/.

[3] https://containernet.github.io/.

2. State of the art

Although several approaches have been considered to automate the security in SDN, only those that use MTD or NFV are reviewed in this section. In this regard, most authors have employed MTD and NFV to build proactive (preventive) defenses against DDoS attacks. For instance, Aydeger et al. [13] proposed an MTD mechanism of random route mutation (RRM) to defend networks from crossfire attacks. They had two objectives: (i) to obfuscate the link map through periodic RRM, and (ii) to mitigate existing crossfire attacks through the migration of the attack traffic towards noncritical routes. They demonstrated the effectiveness of the solution using simulations. Similarly, Ma et al. [14] proposed an MTD-based solution to protect SDN controllers from blind DDoS attacks. They prevented scanning attacks by varying the response time of the flow-rule installation. Additionally, they used a flood filtering module and activated multiple controllers when a blind DDoS attack was detected. Although their experimental results showed convenient performance, this solution could require extensive processing resources and may be non-scalable.

In addition, more advanced strategies have recently been studied that optimize the defense effectiveness and resource usage. In this respect, Zhou et al. [15,16] argued that most MTD solutions ignore defense costs, particularly those related to legitimate users. Thus, they modeled a trilateral game that includes the costs for the attacker, defender, and user. Then they exploited multi-objective Markov Decision Processes (MDP) to find the optimal MTD strategy of a set of existing ones. Through simulations, they showed that their solution reached an acceptable trade-off between effectiveness and cost. In a similar work, Ji et al. [17] presented a multicast routing mutation system against DDoS attacks. They used a heuristic method to find the set of multicast routing trees that satisfies multiple constraints. In addition, they used multiple controllers to achieve an acceptable time complexity. Similarly, Zhou et al. [18] proposed a hybrid solution that combines MTD and cyber deception to protect IoT networks from DDoS attacks. They used a multistage signaling game model led by defenders to model attack-defense processes and analyze the rewards of both players. Subsequently, they used an optimal strategy selection algorithm to reach an appropriate trade-off between effectiveness and cost. The experimental results demonstrated that their solution is resistant to DDoS attacks and provides high-quality, low-delay network services.

It can be noted that, unlike our study, these previous solutions presented in [13–18] did not consider the use of NFV to provide flexibility and scalability to their systems. In addition, unlike our solution, they did not used an automated mechanism to control their frameworks. Furthermore, the targeted attacks are different from those analyzed in our study.

In a more recent approach, the use of combined reactive (detection and mitigation) and proactive (obfuscation and prevention) security solutions has been explored. An example is that of Debroy et al. [19] who proposed a hybrid virtual machine (VM) migration scheme to protect cloud-based applications from DDoS attacks. On the proactive part, they used an optimized mechanism to minimize the migration frequency and select the VM where the target application is migrated. In the reactive part, they used a dedicated VM to keep suspicious connections active, while the actual VMs are hidden from the attackers. Their experiments using simulation demonstrated the effectiveness of the solution. Similarly, Udhaya Prasath et al. [20] presented a hybrid solution against ICMP flooding attacks. Their reactive component used a trained random forest detection model. Additionally, the proactive component changed the virtual host IPs once the attacks were detected. Although their experiments demonstrated a

Table 1
Existing proposals that combine SDN and MTD to defend network from DDoS attacks. Our solution is the first to combine MTD, NFV, and RL to automate the mitigation of slow-rate DDoS attacks. In addition, we provide the code for a framework prototype (FP).

Proposal	MTD	NFV	RL	DDoS	FP	Characteristics
Ma et al. [14]	✓			Blind DDoS		Protects the SDN controller.
Aydeger et al. [13]	✓			Crossfire		Combines reactive and proactive defense.
Aydeger et al. [21]	✓	✓		Crossfire		Uses overlay and mirror networks.
Liu et al. [22]	✓	✓		Proxy harvesting		Uses reverse proxy servers.
Debroy et al. [19]	✓			Slow-rate DDoS		Optimizes frequency and location of VMs.
Zhou et al. [18]	✓			SYN, UDP, ICMP		Protects IoT environments.
Hyder and Fatima [23]	✓	✓		Crossfire		Uses IBN and distributed controllers.
Zhou et al. [15,16]	✓		✓	SYN and UDP.		Uses multi-objective MDP.
Ji et al. [17]	✓			Crossfire		Multi-controller and multi-constraint.
Udhaya Prasath et al. [20]	✓			ICMP flooding		Combines reactive and proactive solutions.
This work	✓	✓	✓	Slow-rate DDoS	✓	Automates the control of DDoS mitigation.

proper response of the solution, changes of the virtual IPs can also affect to legitimate users. It should be noted that our proposed solution also combines reactive (IDS) and proactive mechanisms (MTD) with the advantage that our system leverages NFV and RL to increase the flexibility, scalability, and automation capability of the proposed framework.

Finally, new approaches that use NFV-assisted security solutions against DDoS attacks are emerging. For example, Aydeger et al. [21] presented an SDN/NFV-based framework to solve crossfire attacks. They suggested that adding variability to the typical MTD mechanism of direct route mutation (DRM) is vulnerable to persistent reconnaissance of crossfire attackers. Thus, they proposed adding virtual shadow networks: overlay and mirror networks. Their framework is assisted by NFV to deploy and communicate the shadow networks with the actual network. Their proposal effectively reduced the attack success rate while maintaining an acceptable cost. In a similar approach, Liu et al. [22] presented a fuzzy-based mechanism to detect DDoS attacks. They migrated malicious traffic to reverse proxy servers. In an alternative approach, Hyder and Fatima [23] proposed two mechanisms to solve crossfire attacks. The first solution uses Intent-based networking (IBN) to redirect attack traffic to a shadow server. The second solution uses a domain name system (DNS) to redirect traffic to the shadow server. They did not specify how to detect attackers. In addition, they used multiple controllers to deal with the scalability problem. It should be noted that these three studies demonstrated that NFV offers outstanding advantages over a security solution, such as flexibility, scalability, and cost reduction. Therefore, NFV and the approach of using shadow servers is considered in our framework implementation. Note that these previous authors targeted different attacks than that studied in our work, and they did not automate the control of their frameworks as our solution pursues with the use of RL.

Table 1 summarizes existing studies previously described. Notice that our study is the first to combine MTD, NFV, and RL to mitigate slow-rate attacks. Our main objective is to obtain a flexible, scalable, and automated solution for DDoS detection and mitigation, thus attempting to promote the advancements of the ZSM paradigm. Furthermore, different from previous proposals, we provide the source code of a prototype of our framework.

3. Framework

Fig. 1 shows the proposed framework. The design is based on our previous studies presented in [12,24], where each component of the framework was completely explained. Therefore, in this work, only the new and modified components are thoroughly described.

The framework consists of six modules: Monitor, intrusion detection system (IDS), intrusion prevention system (IPS), MTD, Statistics and Rule Manager, and Reactive Forwarding Routing. First, the Reactive Forwarding Routing module copies the network traffic from the SDN edge switches to the Monitor, by adding some forwarding rules that enforce that each packet is mirrored once (1). Thereafter, the Monitor processes the captured packets to obtain network flows, performs feature selection of the flow statistics, samples a portion of the captured flows, and sends them to the IDS (2).

Following this, the IDS performs variable standardization and principal component analysis (PCA) over the flows and employs a trained model to classify the pre-processed input as suspicious or legitimate. Subsequently, the IDS reports the decisions to the IPS (3), which uses an RL strategy to take the appropriate mitigation actions.

In the next step, the IPS captures the status of the network that is provided by the IDS (3) (suspicious or legitimate connections). Afterward, the IPS takes mitigation actions (4) or instructs the MTD to take them (5). Details of these actions are presented in Sections 5 and 6.

The MTD is designed to migrate traffic from the original server to a shadow server. The MTD needs to solve when, where, and to which shadow server the traffic is migrated. Thus, it uses link statistics, provided by the Statistics and Rule Manager (6), to solve where to migrate the traffic. Likewise, the IPS indicates the MTD when and to which shadow server the traffic is migrated (5).

Finally, the Statistics and Rule Manager module implements the actions received from the IPS (4) and MTD (7). Additionally, this module maintains constant communication with the Reactive Forwarding Routing (8) or other applications to avoid rules collisions or inconsistencies.

Details of IDS, the design of the MTD, and the strategy of IPS are provided in the following sections.

4. Intrusion detection system

This section provides details of the Monitor module and the deep learning model used in the IDS to classify suspicious vs. legitimate traffic.

4.1. Traffic monitoring

In the proposed framework, a data plane-based traffic monitor is presented, where the Monitor module receives copies of the network traffic from edge SDN switches. Under this approach, the controller is not overwhelmed with the network monitoring task, as this process is performed in the data plane. Subsequently, CICFlowMeter [25] is used to generate bidirectional network flows, where each flow has 76 features and is identified by a five-tuple key: Source IP, source port, destination IP, destination port, and protocol. Details of the 76 flow features can be found in [25].

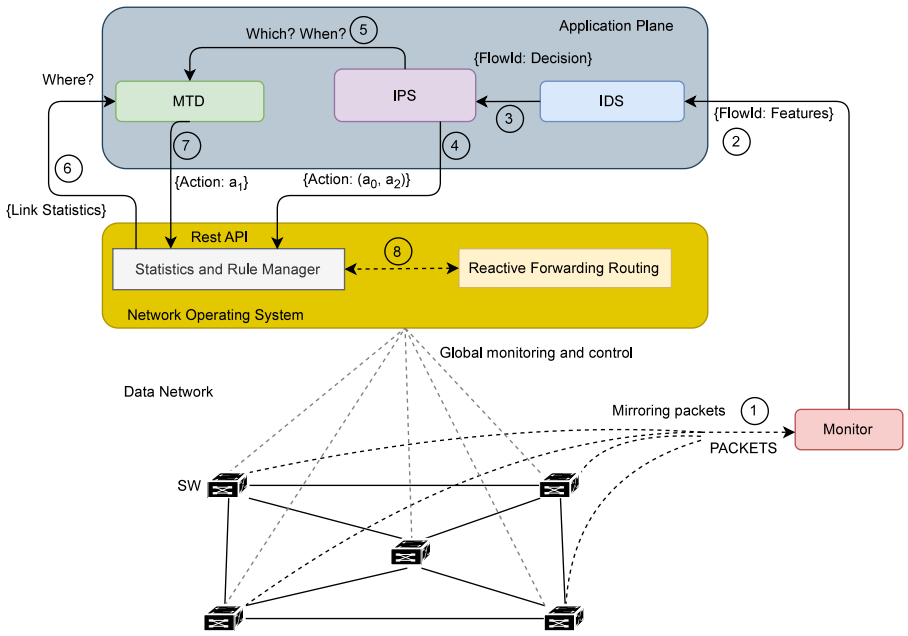

Fig. 1. Proposed framework for autonomous defense against DDoS attacks. Each module of the framework can be independently improved or replaced.

Using all 76 features captured by the CICFlowMeter application is not efficient as some of them do not provide significant information, and others are strongly correlated. Therefore, using the data preprocessing methodology presented in [24], the number of features is reduced to 15 parameters by using feature selection and principal component analysis (PCA). Finally, a flow sampling strategy is included, which selects network flows that are relevant to detect slow-rate DDoS attacks [12]. As a result, the number of flows that reach the IDS represents 30% of the total number of flows existing in the network.

The proposed strategies for data plane-based traffic monitoring and data preprocessing reduce the amount of information that the IDS and the IPS modules need to process, thus increasing the scalability of the security framework.

4.2. Deep learning model

Long short-term memory (LSTM) model is highly effective to detect slow-rate DDoS attacks, as it was demonstrated using the CICDoS2017[4] dataset in [24]. The source code for this model (and some other ML and DL models) to solve slow-rate DDoS attacks is available online[5] Knowing the high performance and robustness of the LSTM model, compared to other ML and DL models, this model was evaluated in our study presented in [12]. In this previous study, an LSTM model was trained and tested using realistic traffic generated in a simulated data center with 97 servers. The training dataset is available online.[6] The obtained LSTM model presented an average detection rate greater than 98% for legitimate and malicious users, when deployed in the simulated data center. It is worth mentioning that this model demonstrated robustness to the variation of number of attackers (up to 24 simultaneous attackers) and the rate of attack connections.

Therefore, the IDS proposed in this study uses this trained and well-tested LSTM model. The IDS receives flows (flow key and flow features) and uses the LSTM model to classify each flow as legitimate or attack. These decisions are saved in the IDS' memory. This information is periodically requested by the IPS. All information transferred to the IPS is immediately deleted from the IDS to avoid memory overflow.

5. Moving target defense

According to Fig. 1, in the design of the MTD mechanism, three questions need to be solved: (a) Which is the traffic to migrate?; (b) When is this traffic migrated?; and (c) where is this traffic migrated? The IPS makes the decision on which traffic is migrated and at what time. Subsequently, the IPS commands the MTD to execute the traffic migration. In short, questions (a) and (b) are solved by the IPS. Section 6 details how the IPS takes these decisions.

The MTD module executes the traffic migration. In this regard, MTD needs to solve Question (c): Find where the traffic is migrated. This question is solved in this section, where the design of the MTD module is presented as follows. First, general details of the MTD mechanism are provided. Subsequently, the representation of link statistics and the computation of the path cost are described, which are used by MTD module to implement the shadow server selection. Finally, the algorithm and tools used to implement the MTD module are reported.

5.1. MTD strategy

The proposed MTD mechanism uses shadow servers that are replicas of the original Web server [26]. The MTD workflow is presented in Fig. 2. In this strategy, a host is initially connected to the original server. When the IPS decides to perform an MTD action, the MTD module subsequently captures path statistics from the Statistics and Rule Manager (S&RM), selects the shadow server, and calculates the route to redirect the traffic of the host to the selected shadow server. Thereafter, the host continues its connection with the shadow server until the IPS decides to execute an alternative action.

A static network address translator (SNAT) is programmed to hide the MTD operation to the host. The SNAT design is presented in Fig. 3. In this design, once the host is redirected to the

4 https://www.unb.ca/cic/datasets/index.html.

5 https://github.com/nmarcelo/MachineLearning_DeepLearning_DDoS_Attack_Detection.git.

6 https://github.com/nmarcelo/Dataset—slow-read-DDoS-attack-in-SDN.git.

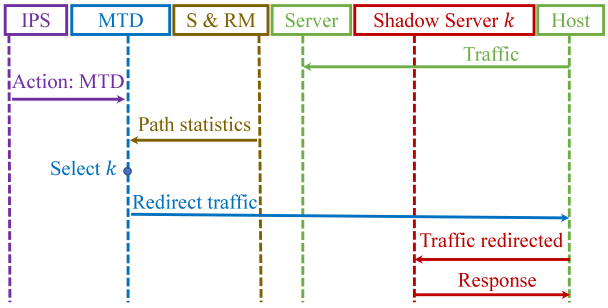

Fig. 2. Workflow of the MTD. S&RM stands for statistics and rule manager.

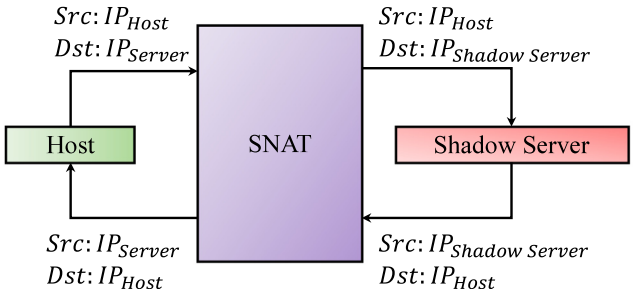

Fig. 3. Static Network Translation (SNAT). From the point of view of the Host, the traffic is sent towards the original server.

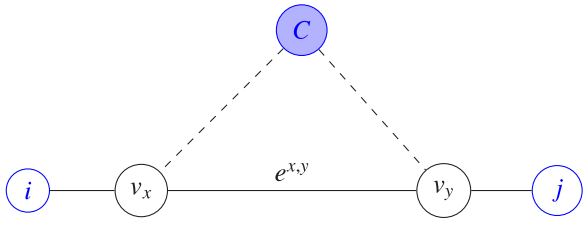

Fig. 4. Network graph representation. The controller C operates the network. Statistics of link $e^{x,y}$ are used by the MTD mechanism. Statistics of the bidirectional connection $c_{ij} = i \Longleftrightarrow j$ are used by the RL-based IPS.

shadow server, the destination IP that corresponds to the server is changed to the shadow server IP. Similarly, traffic that returns to the host is changed, such that its destination IP is replaced by the server IP. Thus, the host believes that it is being served by the original server.

Next, the calculation of the link statistics and path cost are presented, which are used by the MTD module to perform the selection of the kth shadow server.

5.2. Shadow server selection based on link statistics and path cost

To perform the selection of the kth shadow server to migrate the traffic, first the link statistics and path cost need to be calculated as follows.

Fig. 4 shows the network representation used in this study. A network graph $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ is defined, where $\mathbf{V}$ denotes the set of nodes and $\mathbf{E}$ is the set of links among the switches. It is assumed that there are $|V|$ nodes in the network, and therefore $\mathbf{V} = (v_1, v_2, \ldots, v_x, v_y, \ldots, v_{|V|})^\top$. Furthermore, the number of links in a fully connected graph is given by $E = \frac{|V|(|V|-1)}{2}$.

The observation of each link $e^{x,y}$ in the time step t, denoted as $o_t^{x,y}$, is represented by

$$o_t^{x,y} = (t, u^{x,y}, d^{x,y})^\top, \tag{1}$$

where $u^{x,y}$ and $d^{x,y}$ are the link utilization rate and the normalized link delay of $e^{x,y}$.

The link utilization rate is obtained as follows. For a link $e^{x,y}$ at time t_1, $b^{x,y}(t_1)$, which represents the bytes transmitted from node x to y, is measured. Then, after a period $\Delta t = t_2 - t_1$, $b^{x,y}(t_2)$ is recovered. Thus, the utilization rate at time t, $u^{x,y}(t)$, for the link $e^{x,y}$ is given by

$$u^{x,y}(t) = \frac{b^{x,y}(t_2) - b^{x,y}(t_1)}{\Delta t \cdot bw_t(e^{x,y})}, \tag{2}$$

where $bw_t(e^{x,y})$ is the link capacity of $e^{x,y}$.

To estimate the link delay of $e^{x,y}$, four probe packets are sent from the controller C to the devices v_x and v_y. The first two probe packets are enforced to follow the paths C, v_x, C and C, v_y, C. These packets spend $T^{C,x,C}$ and $T^{C,y,C}$ s, respectively. Similarly, the other two probe packets are enforced to follow the paths C, v_x, v_y, C and C, v_y, v_x, C. These packets spend $T^{C,x,y,C}$ and $T^{C,y,x,C}$ s, respectively. The link delay of $e^{x,y}$, $\delta^{x,y}$ is

$$\delta^{x,y} = \max\{T^{C,x,y,C} - \frac{T^{C,x,C} + T^{C,y,C}}{2}, T^{C,y,x,C} - \frac{T^{C,x,C} + T^{C,y,C}}{2}\}. \tag{3}$$

The normalized link delay is obtained as

$$d^{x,y}(t) = \frac{\delta_{x,y}(t)}{d_{max}(t)}, \tag{4}$$

where d_{max} is the maximum of delays of all links in the network at time t.

The path and its cost for a pair of nodes (v_1, v_2) are obtained using the Dijkstra algorithm. The Dijkstra algorithm uses the link cost given by

$$cost(e^{x,y}) = d^{x,y} + u^{x,y}. \tag{5}$$

Finally, to determine to which kth shadow server the traffic of the host i is migrated, the MTD proceeds as follows. First, it is assumed that the host i is connected to v_i, the original server is connected to v_{os}, and the shadow server k is connected to v_k. Second, it is assumed that initially i is served by the original server or by a shadow server l from a group of K shadow servers. Then if the IPS decides to migrate the traffic from i to a new shadow server k, it is chosen probabilistically as

$$P(i,k) = \frac{\frac{1}{cost(e^{v_i,v_k})}}{\sum_{l=1}^{K} \frac{1}{cost(e^{v_i,v_l})}}, \tag{6}$$

where v_l is the node connected to the shadow server l, and the probability to select the shadow server k to migrate the traffic of i is $P(i,k)$. Notice that the shadow server with the lowest path cost (link utilization + link delay) has the highest probability of being selected to migrate the traffic.

5.3. Algorithm of SDN/NFV-assisted MTD

Next, general MTD algorithm is presented, as well as how it interacts with other services in the security framework. Fig. 5 shows the algorithm of the MTD service. First, existing MTD and blocking rules are removed from the SDN switches. Thereafter, the shadow server (SS) selection is performed using (6). Finally, the path from the Host (H) to the Shadow Server (SS) is installed, considering the SNAT service previously described. Note that the IPS decides when to invoke this MTD service, and the Statistics and Rules Manager helps to remove rules, to install paths, and to capture links' statistics used by the MTD algorithm (see Fig. 1).

In the proposed MTD service, NFV helps virtualize network functions (NFs) to make network services much more adaptive

```
procedure MTD_SERVICE({IP_H, IP_S, SSs})
    Remove_Previous_MTD_Rules({IP_H, SSs});
    Remove_Previous_Blocking_Rules({IP_H, IP_S});
    IP_SS ← Shadow_Server_Selection({IP_H, SSs});
    shortest_path ← Get_Shortest_Path({IP_H, IP_SS});
    Install_Path(shortest_path, SNAT({IP_H, IP_S, IP_SS}));
end procedure
```

Fig. 5. MTD service.

and scalable. That is, the implementation of the proposed framework uses NFV to deploy the original and shadow servers. Consequently, the hardware cost is significantly reduced and the independence between application and infrastructure is achieved.

Although NFs can be implemented using virtual machines (VMs), the emerging technology of Lightweight Virtual Machines (LVM) (e.g. Docker[7]), provides better performance [27]. Docker LVM virtualizes the hardware and operating system, and unlike a VM, this new technology is much lighter.

In this work, while the operation of the MTD is performed by the SDN application, the original and shadow servers are deployed using Docker technology. Thus, in addition to the service of traffic migration, servers migration can also be implemented in future work.

6. Intrusion prevention system

This section details the design of the IPS, which is based on RL. Note that the MTD strategy described in Section 5 represents one of the possible actions of the RL agent described in this section. In addition, details of how the decisions of the IPS module are deployed on network devices are explained in this section.

6.1. Reinforcement learning problem

The proposed RL-based IPS uses an agent by connection. To reach the scalability under this approach, the control of the RL agents was presented in [12] using a supervisor system. This work focuses on the design of the zth agent, which manages the bidirectional connection between hosts i and j (see Fig. 4). The states, actions, and rewards for zth agent are described below.

6.1.1. States

In Fig. 4, c_{ij} indicates the bidirectional connection between hosts i and j. Furthermore, $f_{ij}(t'_q)$ is the indicator function that c_{ij} has a suspicious flow ($f_{ij}(t'_q) = 1$) or a legitimate flow ($f_{ij}(t'_q) = 0$) at time $t'_q \in (t_1, t_2)$. If there are W traffic flows in a certain period (t_1, t_2) for c_{ij}, the vector of detection or non-detection events for c_{ij} is

$$\{f_{ij}(t'_1), f_{ij}(t'_2), \ldots, f_{ij}(t'_q), \ldots, f_{ij}(t'_W)\}. \tag{7}$$

Then the instantaneous detection variable $d_{ij}(t)$ is defined as

$$d_{ij}(t) = \begin{cases} 1, & \text{if } \sum\limits_{t'_q \in (t_1,t_2)} f_{ij}(t'_q) > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

Furthermore, g_{ij} defines the number of malicious flows detected in $t'_q \in (t_1, t_2)$ for the bidirectional connection c_{ij} as follows.

$$g_{ij} = \sum_{t'_q \in (t_1,t_2)} f_{ij}(t'_q). \tag{9}$$

[7] https://www.docker.com/.

Table 2
Definition of states. Four states ($s_0 - s_3$) are defined, according to the IDS report (m) and by observing if the connection c_{ij} was previously blocked by the IPS.

Suspicious ($m > M$)	Blocked	State
False	False	s_0
False	True	s_1
True	False	s_2
True	True	s_3

Finally, to quantify how harmful a bidirectional connection is, m_{ij} is defined using (9) and the exponential function as,

$$m_{ij} = \frac{1}{1 + \exp(-(g_{ij} - \eta))}, \tag{10}$$

where η defines how harmful a malicious bidirectional connection is. $\eta = 6$ was used (heuristic definition).

The states are defined using m_{ij} and the status (blocked or unblocked) of c_{ij}, as shown in Table 2. m, defined in (10) as a continuous variable, is reduced to a two-state parameter using a threshold M set by the user. The variable *Blocked* refers to the status of c_{ij}; it indicates whether the connection was previously blocked by the IPS.

Together, the parameters m and *Blocked* define four states as follows:

$$\mathbf{S} = \{s_0, s_1, s_2, s_3\}. \tag{11}$$

6.1.2. Actions

Three actions are considered as follows:

$$\mathcal{A} = \begin{cases} a_0 = \text{Recover connection } c_{ij}, \\ a_1 = \text{MTD}, \\ a_2 = \text{ Block connection } c_{ij}. \end{cases} \tag{12}$$

For a_0, all blocking rules previously installed for c_{ij} are cleaned by the agent. Furthermore, if c_{ij} represents a *host-shadow server* connection, a_0 reconnects i with the original server. For a_1, the agent executes the MTD strategy described in Section 5. Finally, for a_2, the agent installs blocking rules (packet dropping) for c_{ij}.

6.1.3. Reward function

The reward function is defined as

$$r = \rho \; - \; cost(action), \tag{13}$$

where

$$\rho = \begin{cases} 1.0, & \text{if } s_0 \\ 0.0, & \text{if } s_1 \\ -1, & \text{if } s_2 \\ 0.0, & \text{if } s_3 \end{cases} \tag{14}$$

and the cost of the actions is given by

$$cost(action) = \begin{cases} \epsilon, & \text{if } a_1, \\ 0.0, & \text{otherwise.} \end{cases} \tag{15}$$

where a_1 is the action of MTD that consumes the system's resources (shadow servers and execution time), and ϵ, which is a value between (0,1), is the cost of a_1. In this study, a few options were tested for ϵ and $\epsilon = 0.2$ was selected as it provided adequate results. Nevertheless, an exhaustive search of the optimal value of this hyperparameter is proposed in future work. In this regard, interested researchers can explore other values of this hyperparameter using the source code provided to improve the results presented in this work.

Fig. 6 shows the transition graph for the RL problem. Note that s_0 and s_2 are the best state (legitimate connection released) and

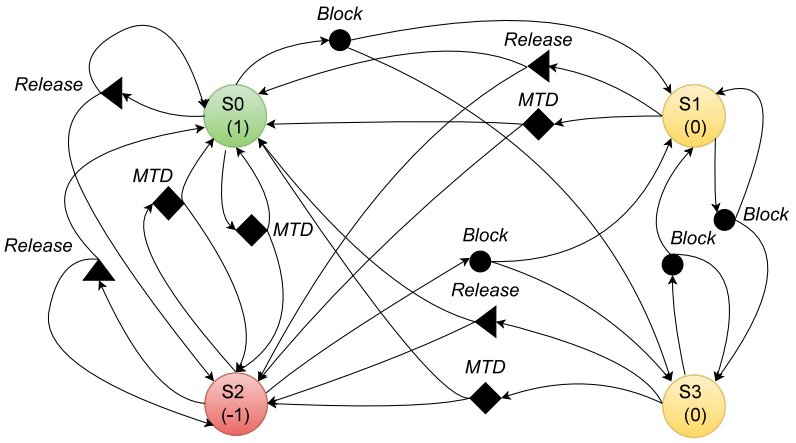

Fig. 6. Transition graph for the RL problem. The colored circles represent the states. The value of each state is shown in parentheses. The filled polygons represent the actions. Notice that the transitions are not deterministic, as they can change if an attack is either detected or stopped.

the worst state (attack connection released), respectively. Further, notice that an attacker can produce a state transition, either if it starts or stops an attack.

6.2. Reinforcement learning solution

Q-Learning is used to solve the RL problem [28], where a state transition table, named Q-table, represents action and values. For each state–action pair (s, a) in the Q table, there is a Q value or $Q(s, a)$ that is updated by

$$Q(s,a) = Q(s,a) + \alpha \cdot \left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right], \tag{16}$$

where s represents the current state, a represents the action executed, s' constitutes the following state, a' indicates the action for the new state, r is the reward obtained for a, $\alpha \in [0, 1]$ is the rate of learning that determines the relevance of newly gained information, and $\gamma \in [0, 1]$ is the discount factor that regulates the significance of future rewards.

6.3. Installation of rules on network devices

The Statistics and Rule Manager module (see Fig. 1) receives the IPS decisions for each bidirectional connection and translates them to flow rules that are installed on the network devices. It is worth mentioning that the IPS is developed using Python. In addition, the Statistics and Rule Manager module, which is built using Java, offers a REST API interface that implements the mitigation actions of the IPS, and the link statistics monitoring that is used by the MTD mechanism.

Furthermore, note that the mitigation rules of the IPS are prioritized over flow rules of other existing applications in the network. For instance, reactive forwarding rules installed by the Reactive Forwarding Routing (see Fig. 1) have lower priority than dropping and traffic migration (MTD) rules. In the case of action a_0, which recovers the connection c_{ij}, the Statistics and Rule Manager module only needs to delete all existing rules for this connection, and the Reactive Forwarding Routing module will automatically create a normal route for i and j, including copying of traffic to the mirroring device for monitoring purposes.

7. Experiments and results

In this section, a series of experiments varying the number of attackers is performed in an simulated testbed to test the performance of the proposed security framework.

Table 3
Parameters of the simulated environment.

Parameter	Value or tool
Virtualized operating system (OS) containing ONOS controller and network simulator Containernet	Ubuntu 20.04.2, Memory: 100 GB, Processor: Intel(R) Xeon (R) W-2155 CPU @ 3.3 GHz x 10.
Web servers using Docker	Image of Ubuntu: *latest version*. Dependencies: *net-tools, iputils-ping, iproute2, build-essential, htop, apache2, wget, systemctl, iperf*. Services running: *iperf, apache2.*
Generation of legitimate traffic	Iperf
Generation of slow-http read DDoS	Slowhttptest.

7.1. Experimental setup

Table 3 shows the parameters and tools used to simulate a testing network. The network emulator Containernet is used to deploy the testing network. Unlike the commonly used Mininet,[8] Containernet allows us to add Docker containers as hosts in an emulated network. In this study, the original and shadow servers are deployed using the Ubuntu Docker image.[9] The dependencies enabled in the Docker containers are listed in Table 3. For instance, *apache2* launches the Apache Web service on the virtual servers.

Furthermore, ONOS controller is used, which is remotely connected to the simulated network. Two Java-based applications are installed on the controller (see Fig. 1): (1) Statistics and Rule Manager and (2) Reactive Forwarding Routing. Finally, legitimate traffic is generated using Iperf,[10] whereas attack traffic is generated using slowhttptest.[11]

Fig. 7 shows the experimental setup. The simulated network contains two attackers and two legitimate hosts, an original server, and two shadow servers. Using this configuration, experiments are deployed to evaluate the proposed automated defense against DDoS attacks.

8 http://mininet.org/.
9 https://hub.docker.com/_/ubuntu.
10 https://iperf.fr/.
11 https://code.google.com/p/slowhttptest/.

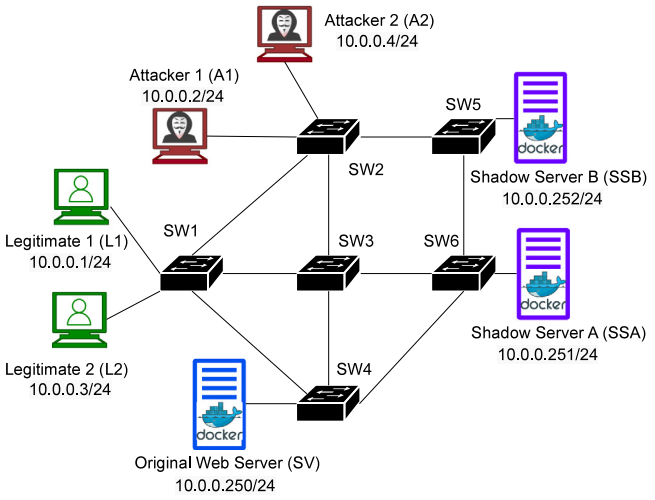

Fig. 7. Experimental setup. The servers are deployed using Docker technology.

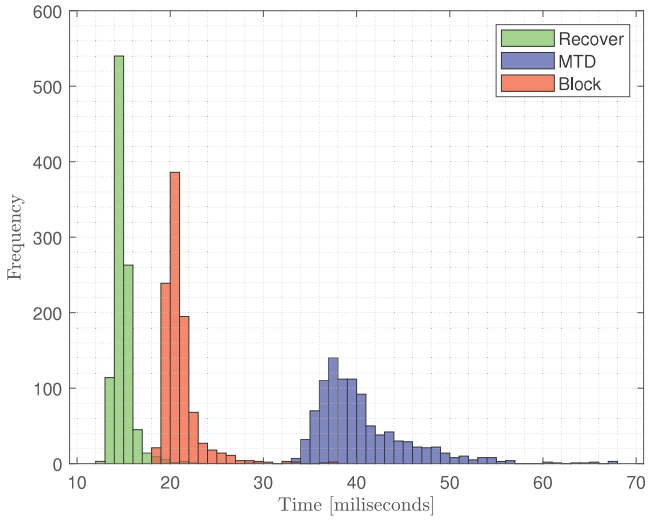

Fig. 8. Histograms of the actions' execution time (measured in 1000 experiments/action). *Recover* and *MTD* have the lowest and the highest execution times, respectively.

7.2. Assessment of the MTD mechanism

Fig. 8 shows the histograms of the execution time for each action of the RL agent. For each action, 1000 experiments were performed. It is observed that *Recover* has a very low execution time, while *MTD* has the highest execution time. Note that the execution the time of the MTD is similar to time obtained in the previous work in [21]. Furthermore, the disadvantage of the execution time of *MTD* was included in the design of the reward function of the IPS, in Section 6, where the cost of the MTD action was higher than those of other actions (see (15)).

7.3. Assessment of the RL-based IPS

Different experiments are performed to assess the performance of the proposed framework, as follows.

7.3.1. Legitimate and attack traffic for one connection

In this experiment, combined legitimate and attack traffic is generated using the connection *A1 - SV* (see Fig. 7). Fig. 9 shows the agent's behaviors of state, action, and reward. Additionally, the traffic (packets/second) of the bidirectional connection is measured at the *eth* port of *A1*. The legitimate and attack periods are $t = 0{-}4000$ s and $t = 0{-}1200$ s, respectively. The framework activates one Q agent to learn the actions that mitigate the attack when needed.

At first, the IDS response is delayed (owing to the flow filtering mechanism), and the Q agent appears to have high rewards. However, once the agent identifies that the connection is under attack (s_2), the reward function hits negative values and the Q agent executes the necessary actions to mitigate the attack. Thus, during the attack period, the action of *Block* (a_2) predominates and the connection is blocked (s_1 or s_3).

Furthermore, once the attack ends, the Q agent tries to recover the connection. However, the agent does not immediately recover the connection, as it needs to guarantee that the connection is now legitimate (s_0). Thus, a period of exploration ($1200 - 2000$ s) is observed before the agent confirms that the connection is legitimate. Regarding the reward, its moving average (MA) function with five samples was included. Therefore, an increasing trend is observed in the reward function.

Finally, the behavior of the traffic (packets/s) shows the effectiveness of the framework response. During the attack period, an elevated number of packets is observed for the connection, which is zeroed with the action taken by the Q agent. Furthermore, after the Q agent recognizes that the connection is now legitimate, the connection is recovered, and the traffic is minimally interrupted by the continuous exploration of the agent.

7.3.2. Legitimate traffic for one connection

This experiment aims to analyze the behavior of the Q agent for legitimate traffic generated using the connection *L1 - SV* for $t = 0 - 4000$ s. Fig. 10 demonstrates that the Q agent minimally affects the network traffic, especially during the initial exploration. The action of *Recover* has the highest frequency throughout the experimental period, maintaining active the legitimate connection (s_0).

7.3.3. Legitimate and attack traffic for multiple connections

In this experiment, two attackers (*A1* and *A2*) and two legitimate hosts (*L1* and *L2*) are used to test the framework (see Fig. 7). Hosts *A1* and *A2* attack the server *SV*, and hosts *L1* and *L2* send legitimate requests to the server *SV*.

This experiment lasted 4000 s. The legitimate hosts maintain the traffic for $t = 0 - 4000$ s. However, the attacks are performed for $t = 0 - 1200$ s. Fig. 11 shows the behavior of the Q agents.

In Fig. 11(a), the actions and rewards for the attack connections are shown. Similarly to the case of a single attacker, the action of *Block* (a_2) has the highest frequency during the attack period. Further, once the attacks stop, the agents explore and resolve that the connections are now legitimate. Thus, the Q agents execute the action of *Recover* to keep the connections active. Finally, the correct learning of the agents is shown in the continuous increase of the reward functions.

The actions and reward functions shown in Fig. 11(b) show the effective response of the Q agents to legitimate connections. Similarly to a single connection, agents prefer the action of *Recover* to keep the legitimate connections active. Additionally, the reward functions demonstrate that the agents correctly learned the appropriate mitigation policy.

7.3.4. Per flow statistics

Table 4 shows the statistics of the number of legitimate and attack flows released, migrated, and blocked for the experiment with multiple connections. First, for each connection c_{ij}, four periods are identified: (i) from the time the attack starts until it is detected by the IDS, (ii) from the time the attack is detected by the

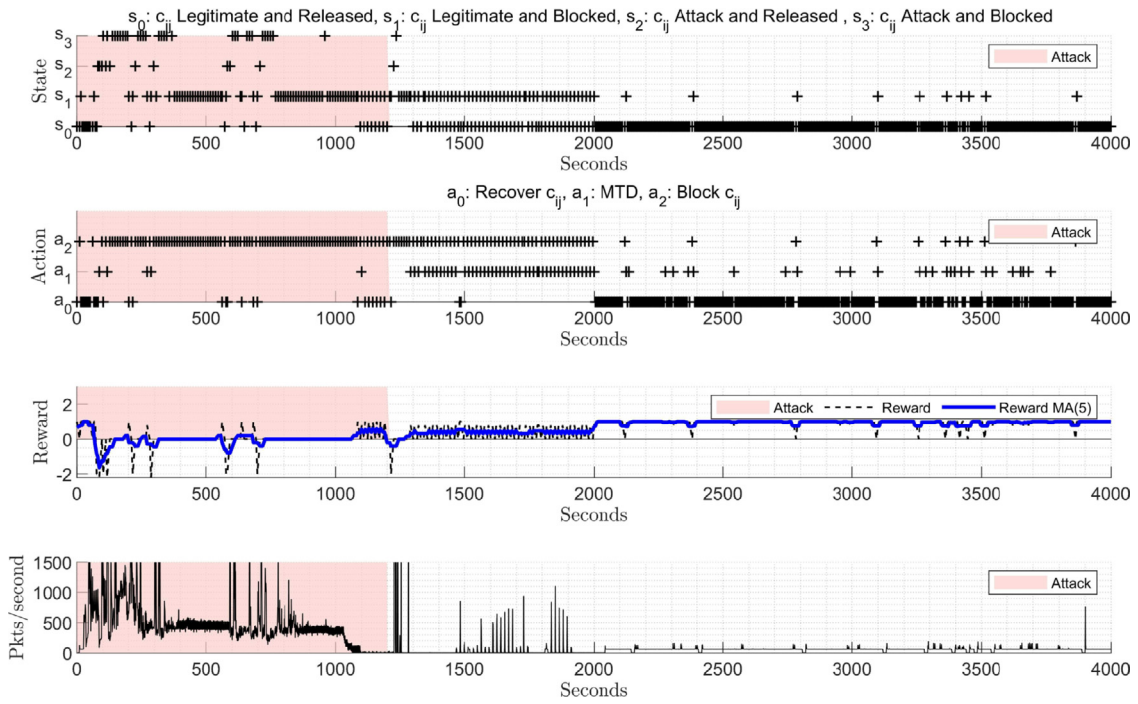

Fig. 9. Q-Learning for connection *A1 - SV*. Legitimate and attack traffic is combined in this connection. The legitimate traffic period is $t = 0 - 4000$ s and the attack period is $t = 0 - 1200$ s. The moving average (MA) of the reward is also displayed to show its trend. Traffic is captured at the *eth* port of *A1*.

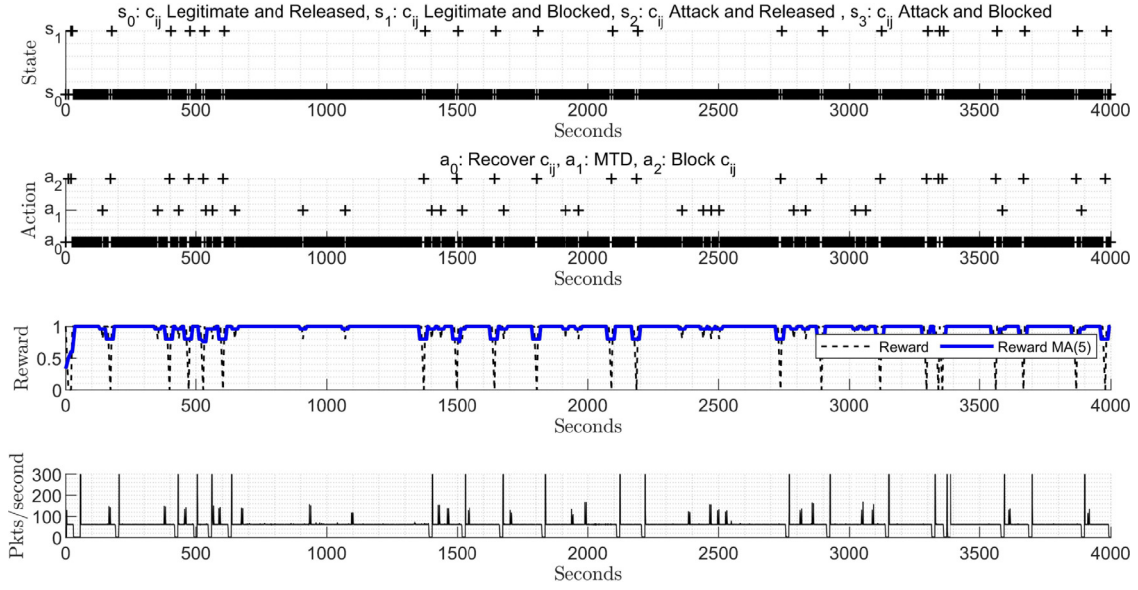

Fig. 10. Q-Learning for connection *L1 - SV*. Only legitimate traffic is generated during $t = 0 - 4000$ s. The moving average (MA) of the reward is also displayed to show its trend. Traffic is captured at the *eth* port of *L1*.

IDS until the attack ends, (iii) from the time the attack ends until the agent ends its exploration, and (iv) from the time the agent ends its exploration until the experiment ends. For legitimate connections c_{ij}, the same periods of the attack connections c_{ij} are considered. Subsequently, the number of events (*Recover*, *MTD*, or *Block*) is counted and averaged for each configured connection (legitimate or attack) and for each period ((i) to (iv)).

The agents learn through all the periods (i) to (iv) in Table 4. During period (i) the agents maintain released most of the flows between the attackers and the original server since the IDS have not reported yet the attack events. Furthermore, note that the agents started to learn in this period, and thus, it is observed that a high percentage of legitimate flows is blocked or sent to a shadow server *k*.

In period (ii), the agents experiment changes in the system states since the IDS starts reporting the attack events. Thus, in this period, the agents learn that legitimate flows need to be released and attack flows must be blocked. Therefore, in period (ii), while most of the attack flows are blocked, only a low percentage of them remains released and a significant percentage of these flows are sent to a shadow server *k*. Under these conditions, most of the resources of the original server are guaranteed to be available to legitimate users. Furthermore, a low percentage of legitimate flows is blocked and a few of these flows are sent to a shadow

Table 4
Statistics of the number of legitimate and attack flows released, migrated, and blocked for the experiment with multiple connections.

Description	(i) Attack starts-Attack is detected		(ii) Attack is detected-Attack ends		(iii) Attack ends-Exploration ends		(iv) Exploration ends-Experiment ends	
Period (s)	0–50.29		50.29–1200		1200–2000		2000-4000	
	Legitimate flows	Attack flows	Legitimate flows	Attack flows	Legitimate flows	Suspicious flows	Legitimate flows	Legitimate flows
i to original server	69.44%	75.69%	90.96%	8.61%	94.08%	54.68%	91.21%	90.48%
i to shadow server *k*	18.75%	6.25%	4.16%	17.55%	3.35%	2.06%	3.16%	4.01%
i blocked	11.81%	18.06%	4.88%	73.08%	2.57%	43.26%	5.06%	4.96%

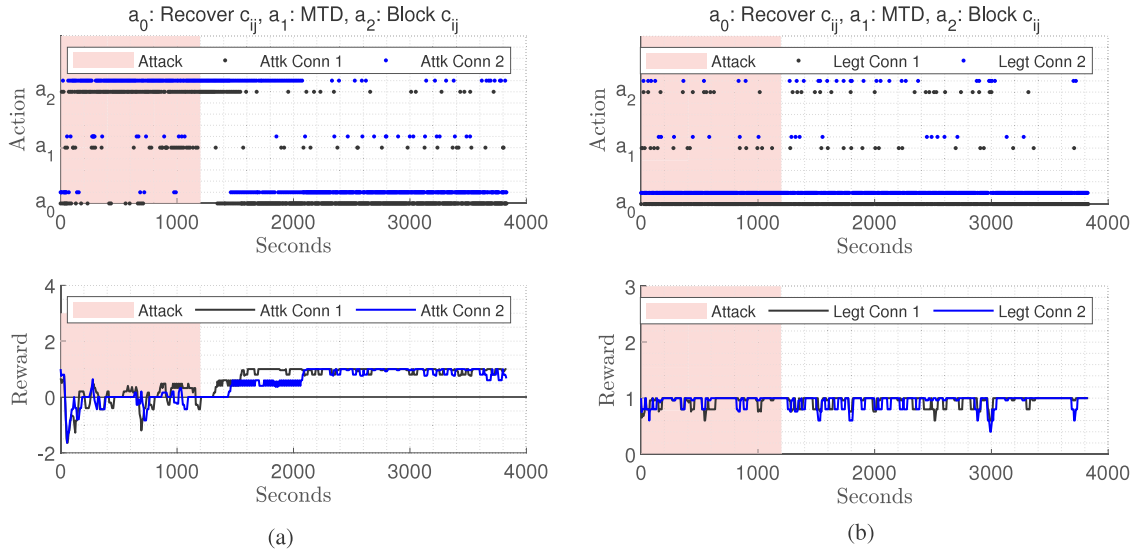

Fig. 11. Q-Learning for multiple connections. (a) Actions and reward for attack connections: Attack connection 1 (Attk Conn 1) from *A1* to *SV*, and attack connection 2 (Attk Conn 2) from *A2* to *SV*. (b) Actions and reward for legitimate connections: Legitimate connection 1 (Legt Conn 1) from *L1* to *SV* and legitimate connection 2 (Legt Conn 2) from *L2* to *SV*.

server *k*. Note that legitimate users do not lose the service when their flows are migrated to the shadows servers.

In period (iii), the attackers have stopped injecting attack flows. Nevertheless, some attack flows remain in the network (switches' queues) and in the chained modules Flow Collector, IDS, IPS. Thus, in this period, the agents explore the states and still observe suspicious behavior of the connection configured as attackers. In this condition, the agents explore by releasing, migrating, and blocking suspicious connection to verify if the attacks are likely to continue or they have already ended. In the case of legitimate flows, the agents learn to keep these flows released.

Finally, in period (iv), all connections are considered legitimate since there are not remaining attack flows in the system. Thus, in this period the agents learn that all connections have legitimate flows and they need to be released.

It is important to note that in each period in Table 4, the agents are constantly learning the correct behavior for each condition. In addition, these statistics are captured from the 30% of the flows filtered by the Flow Collector. Thus, these percentages cannot be considered directly as a measure of the accuracy of the IPS. Nevertheless, these statistics are useful to reveal that the agents selected the proper actions even if the system conditions changed in short periods of time.

7.3.5. System overhead

Fig. 12 depicts the topology used to assess the system performance with a large number of switches and hosts. This topology contains 20 switches, one original server (SV-250), three shadow servers (SS1-251, SS2-252, and SS3-253), and 45 hosts. The hosts are distributed in 30 legitimate users (hosts 1–30) and 15 attackers (hosts 31–45).

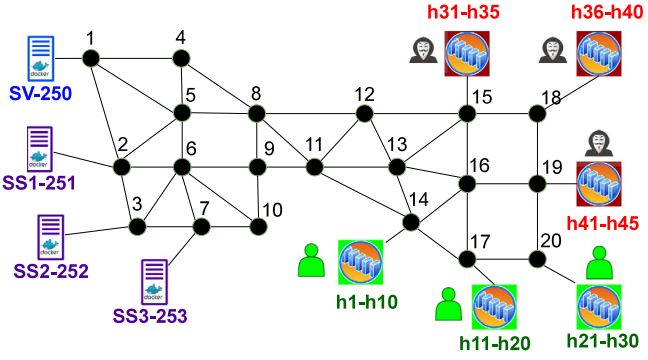

Fig. 12. Topology used to assess the system overhead. SV-250 is the original server with IP 10.0.0.250/24, and SS1-251, SS2-252, and SS3-253 are the shadow servers with IPs 10.0.0.251-253/24. Hosts h1–h30 are legitimate users and hosts h31–45 are attackers.

Fig. 13 shows the performance of the system in terms of actions of the RL agents and the response time to mitigate DDoS attacks. The proposed experiment lasts 500 s. In this experiment, all hosts h1–h45 send legitimate requests to the original server during 0–500 s. During the attack period, which starts at $t = 100$ s and ends at $t = 400$ s, the attackers h31–h45 send malicious requests to the original server.

Fig. 13(a) and (b) present the actions of the RL agents for legitimate and attack connections, respectively. In these figures, the number of actions (Block, MTD, and Recover) of each agent is counted every 120 s. Thus, according to Fig. 13(a), for all legitimate connections, the RL agents prefer Recover action and

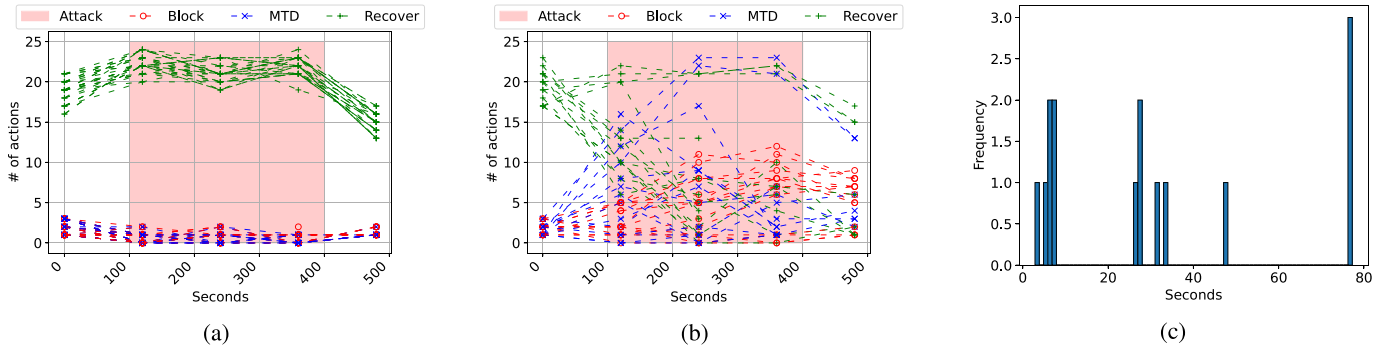

Fig. 13. System performance for 20 switches and 45 hosts. At $t = 100$ s, hosts h31–h45 start to attack (slow http read with an attack connection rate of 30 conn/s) the original server SV-250. The attack ends at $t = 400$ s. (a) Actions of RL agents for legitimate hosts h1–h30. (b) Actions of RL agents for attackers h31–h45. (c) Histogram of the response time of the system to block malicious connections.

maintain the numbers of Drop and MTD actions close to zero during all the experimental period.

In the case of malicious connections represented in Fig. 13(b), the RL agents prefer Drop and MTD actions, during the attack period. Therefore, the attack connections are either blocked or sent to the shadow servers and the resources of the original server are maintained for the legitimate connections. Note that the system maintains the malicious connections blocked after the attack period is finished as the RL agents still consider these connections are malicious.

The histogram of the response time is presented in Fig. 13(c). This response time measures how long it takes to the system to block a malicious connection. It is observed that most values are below 40 s. The mean response time is of 30.80 s.

8. Discussion

This section presents the main findings of this study. In addition, a brief comparison of our solution against other solutions is provided. Finally, as an important contribution of this study, the source code of a prototype of the proposed framework is shared.

8.1. Performance

Experiments using one and multiple attackers allowed a detailed analysis of the Q agents' behavior. For one connection with legitimate and attack traffic, the Q agent was able to find the correct sequence of actions to maximize the reward function. In the same way, for multiple connections, the Q agents learned the solution that leads to mitigate slow-rate attacks and to keep legitimate connections alive. Furthermore, the statistics presented in Table 4 showed that Q agents can quickly adapt to conditions that vary in short periods of time. The agents were able to detect either that attacks were detected and they need to be mitigated or that attackers are now legitimate and they need to be released.

Finally, it should be noted that expanding the number of legitimate or attack connections will lead to the same results presented for one and four simultaneous connections, since the IPS design uses one agent per connection. In this regard, the network topology was expanded and the system was evaluated in terms of actions of the RL agents and the response time to block malicious connections. It was demonstrated that the system properly mitigated attacks in a mean time of 30.80 s, and maintained the legitimate connections released.

8.2. Extended functionality

The advantages of the integrated NFV-assisted MTD mechanism are twofold: (i) For legitimate connections affected by false positives of the IDS, the MTD provides these connections the opportunity to keep them active; and (ii) for suspicious connections, the MTD maintains them connected with a shadow server, and this process is hidden from the attackers. However, the MTD operation has a cost associated with its execution time and use of network resources (shadow servers). Note that this cost is a few milliseconds, so it does not considerably affect the performance of the framework. This cost was included in the design of the RL mechanism.

Furthermore, the addition of an L5 capability (OSI model) was considered to the pool of actions of the RL agent, namely *Reset (RST)*. The action of *RST* consisted in closing TCP threads in the victim server opened by the attackers. Thus, the controller was in charge of building the TCP-RST packets and sending them to the server. However, this strategy needed to store and process additional parameters for each connection, such as the source and destination TCP ports, which significantly increased the processing and storage overheard, making our solution non-scalable. Therefore, this action was not included in the final version of the framework.

8.3. Our framework vs. previous solutions

The emerging paradigms of ZSM and 6G demands of involving mechanisms of ML and DL to automate different networking tasks. In this context, our proposed framework integrates different technologies and intelligent mechanisms to automate the security, and thus significantly contributes to the development of these new paradigms.

Previous works either solved the security problem partially (by only designing the IDS) or they did not seize the advantages of ML and DL mechanisms to automate the mitigation of DDoS attacks. Hyder and Ismail [26], Tao et al. [27] and Rezapour and Tzeng [4] proposed relevant ideas of using MTD, NFV, and RL to defend network of DDoS, but they cannot be considered automated nor comprehensive solutions, since they focused on demonstrating the feasibility of applying different mechanisms or technologies to solve diverse tasks. Nonetheless, the technologies used in these works inspired the design of some modules presented in our framework. To the best of our knowledge, this work represents the first endeavour to comprehensively solve the security problem in SDNs by automating the detection and mitigation tasks of slow-rate DDoS attacks and by integrating novel NFV technologies.

8.4. Prototype of the framework

The source code for a prototype of the framework is shared,[12] so it can be used or improved by interested researchers. We

[12] https://github.com/nmarcelo/SDNFV_Framework_DDoS_Solution.git.

will continuously improve the current version of the software by adding functionality to the application.

9. Conclusion and future work

The proposed framework demonstrated its effectiveness in mitigating slow-rate DDoS attacks. The addition of the NFV-assisted MTD mechanism allowed us to increase the network performance since legitimate connections are less affected by false positives and attack connections are deviated to shadow servers. In addition, the RL mechanism brought the intelligent decision-making capability to the network management system, which is the key in the development of the 6G technology and ZSM paradigm. Finally, when the network topology was expanded, the system maintained its performance.

For future work, we plan to improve the scalability of the proposed framework by using NFV, such as implementing virtual server migration. Moreover, the optimization of the hyperparameters of the proposed RL algorithm is proposed for future work. Furthermore, P4 will be considered to capture and process packets at the data plane (as an improvement of the Flow Collector) with higher speed, which will enhance the scalability our solution. Finally, demonstration of the framework on a physical testbed is also considered for future work.

CRediT authorship contribution statement

Noe M. Yungaicela-Naula: Conceptualization, Methodology, Investigation, Software, Validation, Writing – original draft. **Cesar Vargas-Rosales:** Conceptualization, Methodology, Reviewing and editing. **Jesús A. Pérez-Díaz:** Conceptualization, Methodology, Reviewing and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work was partially supported by FRIDA (Fondo Regional para la Innovación Digital en America Latina *y* el Caribe); the project "Red temática Ciencia *y* Tecnología para el desarrollo (CYTED) 519RT0580" by the Ibero-American Science and Technology Program for development CYTED; a 2020 Seed Fund award from Tecnologico de Monterrey & CITRIS and the Banatao Institute at the University of California, United States; the School of Engineering and Sciences at Tecnologico de Monterrey; and the Telecommunications Research Group at Tecnologico de Monterrey, United States.

References

[1] R. Hummel, R. Dobbins, C. Conrad, S. Bjarnson, J. Kristoff, D. Culver, NETSCOUT Threat Intelligence Report: Findings from 2nd Half 2021, Technical Report, NETSCOUT, 2022.

[2] J.A. Herrera, J.E. Camargo, A survey on machine learning applications for software defined network security, in: International Conference on Applied Cryptography and Network Security, Springer, 2019, pp. 70–93, http://dx.doi.org/10.1007/978-3-030-29729-9_4.

[3] P. Wang, L.T. Yang, X. Nie, Z. Ren, J. Li, L. Kuang, Data-driven software defined network attack detection: State-of-the-art and perspectives, Inform. Sci. 513 (2020) 65–83, http://dx.doi.org/10.1016/j.ins.2019.08.047.

[4] A. Rezapour, W.-G. Tzeng, RL-shield: Mitigating target link-flooding attacks using SDN and deep reinforcement learning routing algorithm, IEEE Trans. Dependable Secure Comput. (2021) 1, http://dx.doi.org/10.1109/TDSC.2021.3118081.

[5] S. Sengupta, A. Chowdhary, A. Sabur, A. Alshamrani, D. Huang, S. Kambhampati, A survey of moving target defenses for network security, IEEE Commun. Surv. Tutor. 22 (3) (2020) 1909–1941, http://dx.doi.org/10.1109/COMST.2020.2982955.

[6] A. Aydeger, N. Saputro, K. Akkaya, A moving target defense and network forensics framework for ISP networks using SDN and NFV, Future Gener. Comput. Syst. 94 (2019) 496–509, http://dx.doi.org/10.1016/j.future.2018.11.045.

[7] Y. Siriwardhana, P. Porambage, M. Liyanage, M. Ylianttila, AI and 6G security: Opportunities and challenges, in: 2021 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit), 2021, pp. 616–621, http://dx.doi.org/10.1109/EuCNC/6GSummit51104.2021.9482503.

[8] M. Liyanage, Q.-V. Pham, K. Dev, S. Bhattacharya, P.K.R. Maddikunta, T.R. Gadekallu, G. Yenduri, A survey on Zero touch network and Service Management (ZSM) for 5G and beyond networks, J. Netw. Comput. Appl. 203 (2022) 103362, http://dx.doi.org/10.1016/j.jnca.2022.103362.

[9] J. Gallego-Madrid, R. Sanchez-Iborra, P.M. Ruiz, A.F. Skarmeta, Machine learning-based zero-touch network and service management: a survey, Digit. Commun. Netw. 8 (2) (2022) 105–123, http://dx.doi.org/10.1016/j.dcan.2021.09.001.

[10] N.M. Yungaicela-Naula, C. Vargas-Rosales, J.A. Pérez-Díaz, M. Zareei, Towards security automation in software defined networks, Comput. Commun. 183 (2022) 64–82, http://dx.doi.org/10.1016/j.comcom.2021.11.014.

[11] P. Goransson, C. Black, T. Culver, Software Defined Networks: A Comprehensive Approach, Morgan Kaufmann, 2016.

[12] N.M. Yungaicela-Naula, C. Vargas-Rosales, J.A. Pérez-Díaz, D.F. Carrera, A flexible SDN-based framework for slow-rate DDoS attack mitigation by using deep reinforcement learning, J. Netw. Comput. Appl. 205 (2022) 103444, http://dx.doi.org/10.1016/j.jnca.2022.103444.

[13] A. Aydeger, N. Saputro, K. Akkaya, M. Rahman, Mitigating crossfire attacks using SDN-based moving target defense, in: 2016 IEEE 41st Conference on Local Computer Networks (LCN), 2016, pp. 627–630, http://dx.doi.org/10.1109/LCN.2016.108.

[14] D. Ma, Z. Xu, D. Lin, Defending blind ddos attack on SDN based on moving target defense, in: J. Tian, J. Jing, M. Srivatsa (Eds.), International Conference on Security and Privacy in Communication Networks, Springer International Publishing, Cham, 2015, pp. 463–480, http://dx.doi.org/10.1007/978-3-319-23829-6_32.

[15] Y. Zhou, G. Cheng, S. Jiang, Y. Hu, Y. Zhao, Z. Chen, A cost-effective shuffling method against DDoS attacks using moving target defense, in: Proceedings of the 6th ACM Workshop on Moving Target Defense, Association for Computing Machinery, New York, NY, USA, 2019, pp. 57–66, http://dx.doi.org/10.1145/3338468.3356824.

[16] Y. Zhou, G. Cheng, S. Jiang, Y. Zhao, Z. Chen, Cost-effective moving target defense against DDoS attacks using trilateral game and multi-objective Markov decision processes, Comput. Secur. 97 (2020) 101976, http://dx.doi.org/10.1016/j.cose.2020.101976.

[17] W. Ji, S. Yang, B. Zhang, T. Zhang, Y. Lian, C. Xu, Multi-domain multicast routing mutation scheme for resisting DDoS attacks, in: 2022 International Wireless Communications and Mobile Computing (IWCMC), 2022, pp. 142–147, http://dx.doi.org/10.1109/IWCMC55113.2022.9824529.

[18] Y. Zhou, G. Cheng, S. Yu, An SDN-enabled proactive defense framework for DDoS mitigation in IoT networks, IEEE Trans. Inf. Forensics Secur. 16 (2021) 5366–5380, http://dx.doi.org/10.1109/TIFS.2021.3127009.

[19] S. Debroy, P. Calyam, M. Nguyen, R.L. Neupane, B. Mukherjee, A.K. Eeralla, K. Salah, Frequency-minimal utility-maximal moving target defense against DDoS in SDN-based systems, IEEE Trans. Netw. Serv. Manag. 17 (2) (2020) 890–903, http://dx.doi.org/10.1109/TNSM.2020.2978425.

[20] M. Udhaya Prasath, B. Sriram, P. Prakashkumar, V. Vetriselvi, DDoS mitigation in SDN using MTD and behavior-based forwarding, in: H.S. Saini, R.K. Singh, M. Tariq Beg, R. Mulaveesala, M.R. Mahmood (Eds.), Innovations in Electronics and Communication Engineering, Springer Singapore, Singapore, 2022, pp. 373–380, http://dx.doi.org/10.1007/978-981-16-8512-5_40.

[21] A. Aydeger, N. Saputro, K. Akkaya, Utilizing NFV for effective moving target defense against link flooding reconnaissance attacks, in: MILCOM 2018 - 2018 IEEE Military Communications Conference (MILCOM), 2018, pp. 946–951, http://dx.doi.org/10.1109/MILCOM.2018.8599803.

[22] C.-C. Liu, B.-S. Huang, C.-W. Tseng, Y.-T. Yang, L.-D. Chou, SDN/NFV-Based moving target ddos defense mechanism, in: F. Saeed, N. Gazem, F. Mohammed, A. Busalim (Eds.), Recent Trends in Data Science and Soft Computing, Springer International Publishing, Cham, 2019, pp. 548–556, http://dx.doi.org/10.1007/978-3-319-99007-1_51.
[23] M.F. Hyder, T. Fatima, Towards crossfire distributed denial of service attack protection using intent-based moving target defense over software-defined networking, IEEE Access 9 (2021) 112792–112804, http://dx.doi.org/10.1109/ACCESS.2021.3103845.
[24] N.M. Yungaicela-Naula, C. Vargas-Rosales, J.A. Perez-Diaz, SDN-based architecture for transport and application layer DDoS attack detection by using machine and deep learning, IEEE Access 9 (2021) 108495–108512, http://dx.doi.org/10.1109/ACCESS.2021.3101650.
[25] CIC Flow Meter, 2020. Canadian institute for cybersecurity, URL: https://github.com/CanadianInstituteForCybersecurity/CICFlowMeter.
[26] M.F. Hyder, M.A. Ismail, INMTD: Intent-based moving target defense framework using software defined networks, Eng., Technol. Appl. Sci. Res. 10 (1) (2020) 5142–5147, http://dx.doi.org/10.48084/etasr.3266.
[27] X. Tao, F. Esposito, A. Sacco, G. Marchetto, A policy-based architecture for container migration in software defined infrastructures, in: 2019 IEEE Conference on Network Softwarization (NetSoft), IEEE, 2019, pp. 198–202, http://dx.doi.org/10.1109/NETSOFT.2019.8806659.
[28] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 1998.

Noe M. Yungaicela-Naula received his B.Sc degree in Electronic and Telecommunication Engineering from the Universidad de Cuenca (Cuenca, Ecuador), in 2015, and his M.Sc. degree in Intelligent Systems at Tecnologico de Monterrey in 2018. From November 2017 to March 2018, he was a Visiting Scholar at the Concordia University, Montreal, QC, Canada. Nowadays, he is pursuing his Ph.D. degree at the Tecnologico de Monterrey. His research interests are machine learning, deep reinforcement learning, network security, and software defined networking.

Cesar Vargas-Rosales (M'89–SM'01) has a Ph.D. and a M.Sc. in Electrical Engineering from Louisiana State University in Communications and Signal Processing. Dr. Vargas is a member of the Mexican National Researchers System (SNI), the Mexican Academy of Science (AMC), and the Academy of Engineering of Mexico. He is an Associate Editor of IEEE Access and International Journal of Distributed Sensor Networks. He is a Senior member of the IEEE, the IEEE Communications Society Monterrey Chapter Chair, and the Faculty advisor of the IEEE-HKN Lambda-Rho Chapter at Tecnologico de Monterrey. He was also the Technical Program Chair of the IEEE Wireless Communications and Networking Conference (IEEE WCNC). He is the co-author of the book Position Location Techniques and Applications (Academic Press/Elsevier). His research interests are personal communications, 5G/6G, Cognitive Radio, MIMO systems, stochastic modeling, traffic modeling, Intrusion/anomaly detection in networks, position location, interference, network and channel coding, and optimum receiver design.

Jesus Arturo Perez-Diaz obtained his B.Sc. degree in computer science from the Autonomous University of Aguascalientes in 1995, where he received the best student award. He received his Ph.D. degree in New Advances in Computer Science Systems from the Universidad de Oviedo in 2000. He became a full associate professor at University de Oviedo from 2000 to 2002. He was recognized by the COIMBRA group as one of the best young Latin-American researchers in 2006 and received a research stay at Louvain le nouveau University in Belgium. He has been awarded by the CIGRE and by Intel for the development of innovative systems. Currently, he is a researcher and professor at Tecnologico de Monterrey – Campus Querétaro, Mexico and member of the Mexican National Researchers System. His research field focus in cyber-security in SDN and design of communications protocols where he has supervised several master and Ph.D. theses in the field.